



University of Idaho

Department of Computer Science

CS 4727/5727
Adversarial
Machine Learning

Dr. Alex Vakanski



Lecture 10

AML in Cybersecurity – Part II: Malware Detection and Classification



Lecture Content

10.3 Malware Detection and Classification

10.3.1 Malware Analysis Systems

10.3.1.1 Static Malware Analysis Systems

10.3.1.2 Dynamic Malware Analysis Systems

10.3.2 ML Models for Malware Classification

10.3.2.1 Static Features

10.3.2.2 Dynamics Features

10.3.3 Adversarial Attacks on ML-based Malware Classifiers

10.3.3.1 Attacks on Traditional ML Malware Classifiers

10.3.3.2 Attacks on Deep Learning Malware Classifiers

Student Presentation by Niranjana Bekoju

10.3.3.3 Severi (2021) Explanation-Guided Backdoor Poisoning Attacks Against Malware Classifiers

Malware Detection and Classification

10.3 Malware Detection and Classification

- **Mal**icious soft**ware** is also known as malware
- **Malware** is any kind of software that is specifically designed to disrupt, damage, or gain unauthorized access to a computer system or network
 - Malware is constantly evolving and presents significant threat to computer systems
 - Forces security analysts to keep pace by improving cyber defenses
- Automated malware detection versus classification
 - **Malware detection** systems: predict whether an executable file is malware or not a malware
 - Output: 1 or 0
 - **Malware classification** systems: predict the malware type of an executable
 - Output: 1 to N , where N is the number of different malware families
 - I.e., malware classification systems differentiate between different kinds of malware (virus, adware, or Trojan), in order to provide a better understanding of their capabilities

Malware Categories

10.3 Malware Detection and Classification

- Depending on the purposes, malware can be divided into various categories
 - **Virus**: attaches itself to a program and infects a device
 - **Worm**: self-replicates and propagates copies of itself to other devices over a network
 - **Adware**: generates/displays unsolicited online advertisements on user's screen
 - **Ransomware**: locks down an infected device, and demands payment to unlock it
 - **Backdoor**: allows unauthorized access to functionality
 - **Trojan**: a class of backdoor malware disguised as legitimate software, to trick users into installing it
 - **Bot**: distributes malware to other devices, and it is typically part of a network (botnet)
 - **Keyloggers**: captures keystrokes
 - **Rootkit**: gains root-level access to conceal the existence of other malware
 - **Logic bomb**: explodes when a condition occurs

Malware Analysis Systems

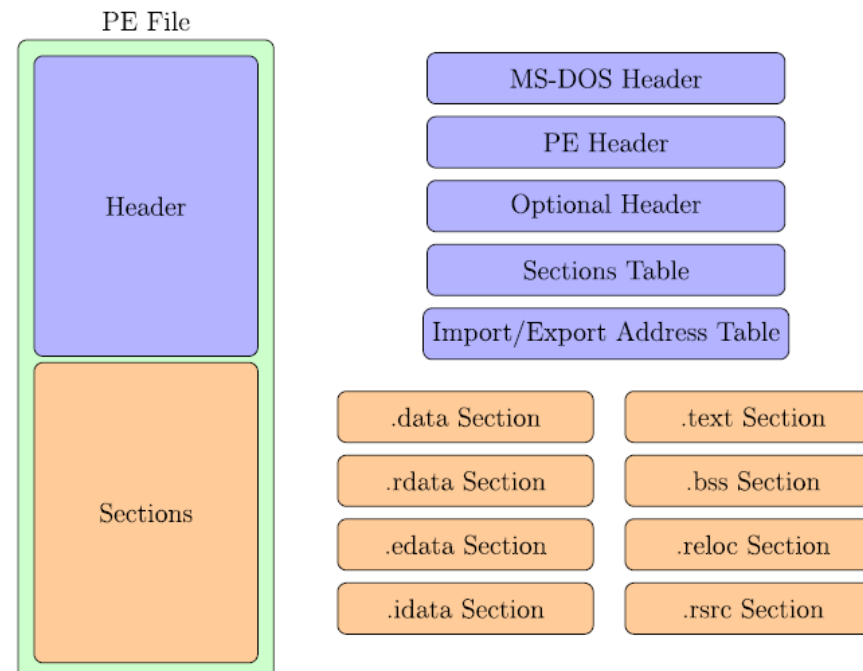
10.3.1 Malware Analysis Systems

- *Malware analysis* involves dissecting malware to understand how it works, and determine its functionality, origin, and potential impact
 - Malware analysis is essential for any business and infrastructure that responds to cybersecurity incidents
- Malware analysis systems can be classified into two broad categories
 - *Static analysis systems* (pre-execution analysis)
 - Process malware without running it, and extract features to be used for malware detection and classification
 - *Dynamic analysis systems* (post-execution analysis)
 - It involves running the malware either in a physical or virtual environment, and searching for indicators of malicious activities
- Some references also add a class of *hybrid analysis systems*, that combine static and dynamic analysis

Portable Executable (PE) File Format

10.3.1 Malware Analysis Systems

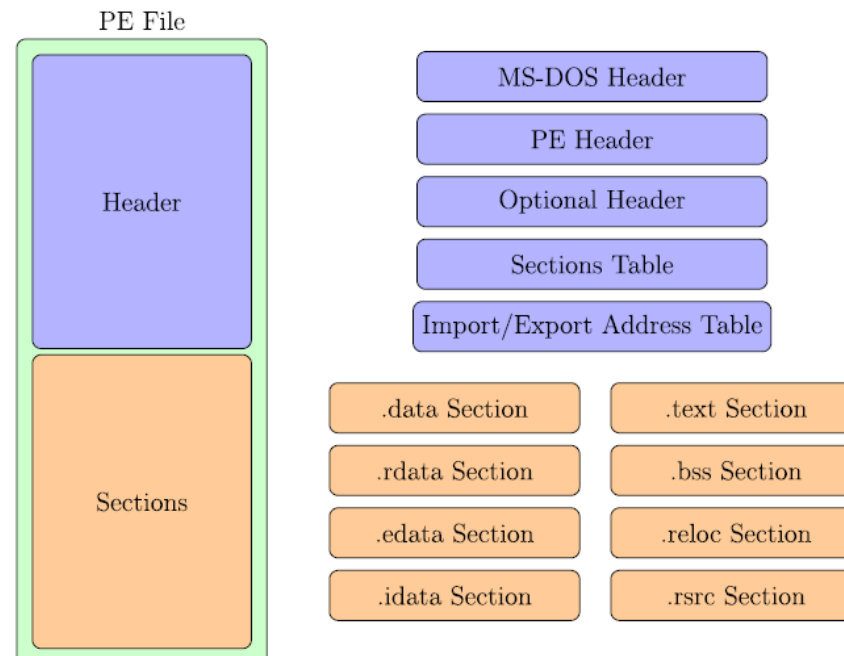
- In Windows systems, *Portable Executable* (PE) is a file format for executables
 - Analogous formats are Executable and Linkable Format (ELF) in Linux and Unix systems, and Mach-O in macOS and iOS
 - Most existing malware targets Windows systems
- A PE file consists of a number of a *header* and *sections* that inform the Windows OS how to manage the executable file



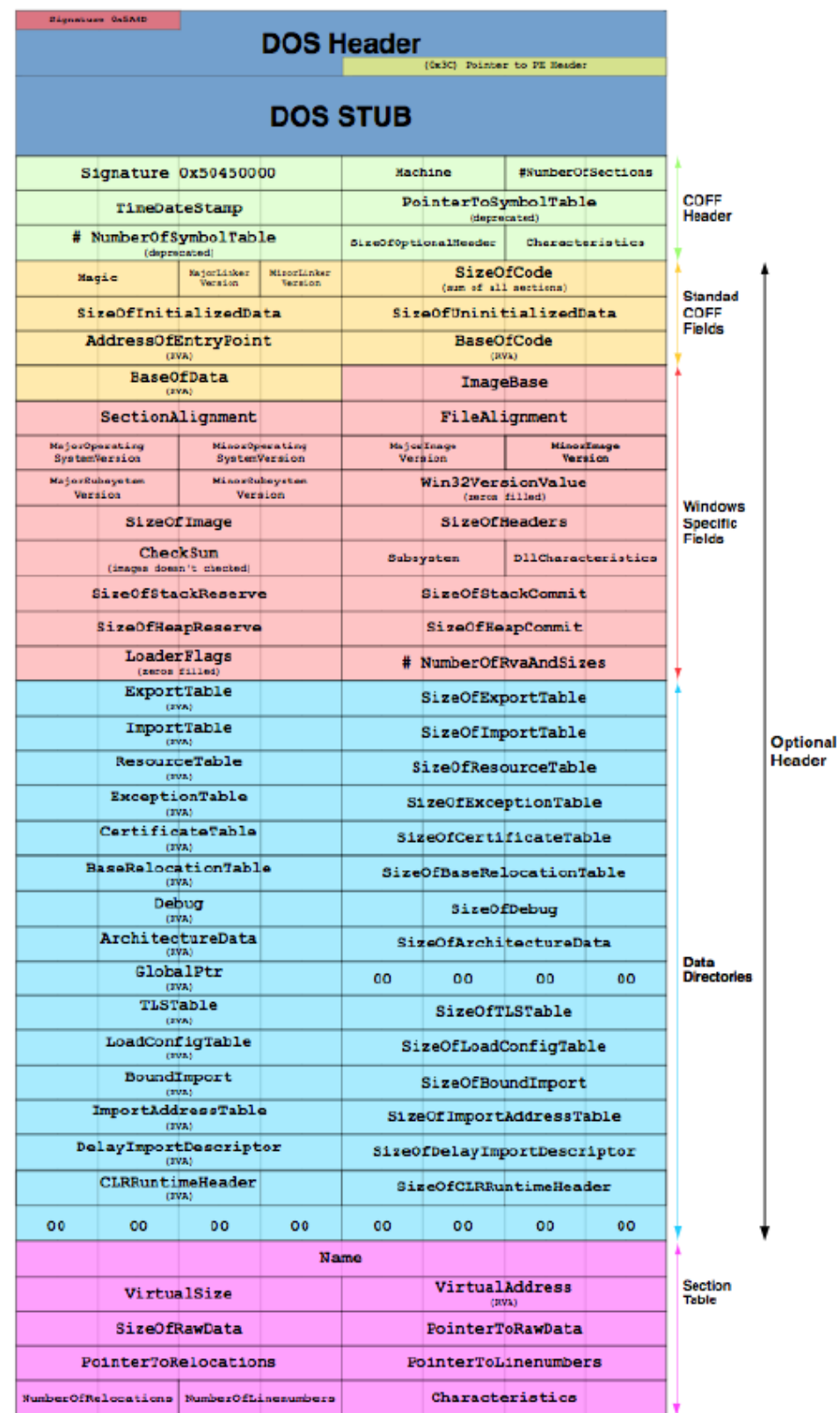
Portable Executable (PE) File Format

10.3.1 Malware Analysis Systems

- PE file format
 - **PE file header**
 - The header of the PE file is composed of additional headers (MS-DOS, PE, and Optional Header) and various tables and fields (Sections Table, Import/Export Address Table)
 - **PE file sections**
 - The sections are either code sections (containing machine instructions), data sections (holding variables and constants), or resource sections (holding embedded fonts, images, etc.)



- The format for a 32-bit PE file header is shown in the figure
- Note the structure from top to bottom:
 - MS-DOS Header with the DOS Stub
 - PE Header, i.e., COFF (Common Object File Format) Header
 - Optional Header
 - Sections Table
- Detailed description of the PE format can be found at <https://docs.microsoft.com/en-us/windows/win32/debug/pe-format>



- Extracted features from a PE file for ML analysis in the EMBER dataset
- Note again the header, optional header, imports and exports, sections, etc.

```

"sha256": "000185977be72c8b007ac347b73ceb1ba3e5e4dae4fe98d4f2ea92250f7f580e",
"appeared": "2017-01",
"label": -1,
"general": {
  "file_size": 33334,
  "vsize": 45056,
  "has_debug": 0,
  "exports": 0,
  "imports": 41,
  "has_relocations": 1,
  "has_resources": 0,
  "has_signature": 0,
  "has_tls": 0,
  "symbols": 0
},
"header": {
  "coff": {
    "timestamp": 1365446976,
    "machine": "I386",
    "characteristics": [ "LARGE_ADDRESS_AWARE", ..., "EXECUTABLE_IMAGE" ]
  },
  "optional": {
    "subsystem": "WINDOWS_CUI",
    "dll_characteristics": [ "DYNAMIC_BASE", ..., "TERMINAL_SERVER_AWARE" ],
    "magic": "PE32",
    "major_image_version": 1,
    "minor_image_version": 2,
    "major_linker_version": 11,
    "minor_linker_version": 0,
    "major_operating_system_version": 6,
    "minor_operating_system_version": 0,
    "major_subsystem_version": 6,
    "minor_subsystem_version": 0,
    "sizeof_code": 3584,
    "sizeof_headers": 1024,
    "sizeof_heap_commit": 4096
  }
},
"imports": {
  "KERNEL32.dll": [ "GetTickCount" ],
  ...
},
"exports": [],
"section": {
  "entry": ".text",
  "sections": [
    {
      "name": ".text",
      "size": 3584,
      "entropy": 6.368472139761825,
      "vsize": 3270,
      "props": [ "CNT_CODE", "MEM_EXECUTE", "MEM_READ" ]
    },
    ...
  ]
},
"histogram": [ 3818, 155, ..., 377 ],
"byteentropy": [ 0, 0, ..., 2943 ],
"strings": {
  "numstrings": 170,
  "avlength": 8.170588235294117,
  "printabledist": [ 15, ..., 6 ],
  "printables": 1389,
  "entropy": 6.259255409240723,
  "paths": 0,
  "urls": 0,
  "registry": 0,
  "MZ": 1
},
}

```

Static Analysis Systems

10.3.1.1 Static Malware Analysis Systems

- **Static analysis** provides information about the functionality of the file, and it produces a set of signature features (without executing the file)
 - The extracted information is used to predict whether the file is malicious software
 - The disadvantage of static analysis is that the “true features” of the code may be missed
- Static analysis can include:
 - **Analyzing PE header and sections**
 - PE header provides information about linked libraries and imported/exported functions, as well as contains metadata about the executable file
 - Strings of characters can contain references to modified files or accessed file paths by the executable (e.g., URLs, domain names, IP addresses, names of loaded DLLs, registry keys, etc.)
 - Packed/encrypted code that is used by malware developers to make their manipulated files more difficult to analyze can be used for malware detection
 - **Disassembling the program** – translating machine code into assembly language code
 - Load the executable into a disassembler to translate it into assembly language, and obtain a better understanding of what the program does

Dynamic Analysis Systems

10.3.1.2 Dynamic Malware Analysis Systems

- *Dynamic analysis* involves executing the program and monitoring its behavior
 - It is typically performed when all available static analysis techniques have been exhausted
- Dynamic analysis is run in a safe environment on dedicated physical or virtual machines (in order not to expose the users' system to unnecessary risks)
 - *Physical machines* are set up on isolated networks, disconnected from the Internet or any other network, to prevent malware from spreading
 - *Virtual machines* emulate the functionality of a physical computer, where the OS running on the virtual machine is isolated from the host OS
 - One limitation is that some malware can detect when they are running in a virtual machine, and they will execute differently than when in a physical machine
 - A related term is *sandbox*, referring to a physical or virtual environment for running malware, which isolates executables from other system resources and applications
 - Although sandboxes share characteristics with physical and virtual machines, sandboxes can be more limited (e.g., they can run in the browser), while physical and virtual machines always act as a complete system
 - For example, online sandboxes are websites where one can submit a sample file and receive a report about its behavior

ML-based Malware Classification

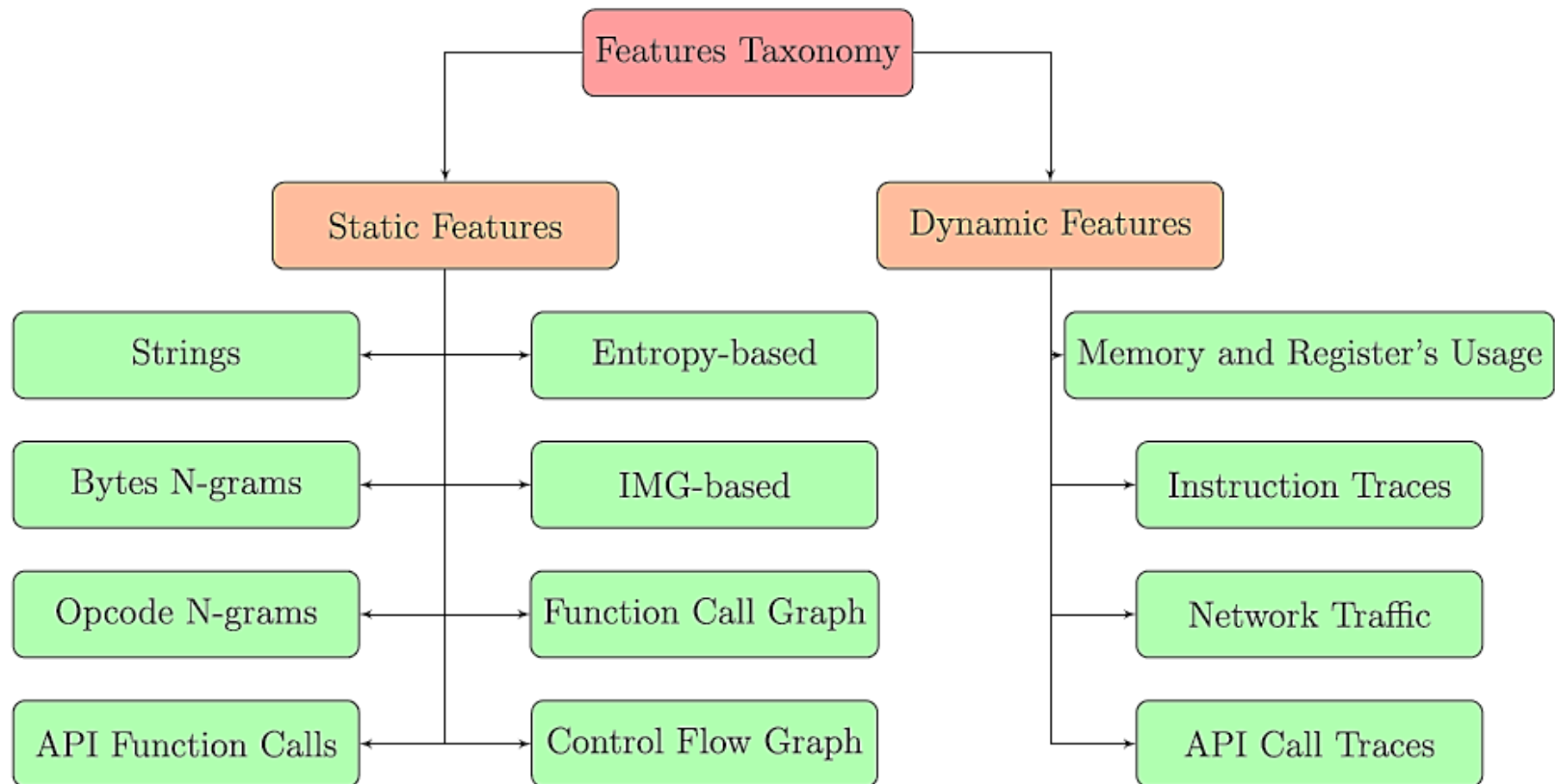
10.3.2 ML Models for Malware Classification

- *ML-based systems for malware detection and classification* employ a set of extracted features from executable files
 - As we mentioned before, unlike the ML models in Computer Vision that employ the intensity of image pixels for image classification, segmentation, or object detection tasks, ML models for malware classification employ a great variety of different features
 - The extracted features are used for training an ML model, and understandably, the trained model is afterwards used for detection or classification of new files
 - In the remainder of the lecture, we will use the term “classification” or “classifier” to describe both ML models for detection and classification of malware
- Based on the inputs used for malware classification, the ML-based systems can be broadly categorized into:
 - *Raw-binary classifiers* - use raw byte content from executables as input features
 - *Feature-based classifiers* - use either *static* or *dynamic* features, obtained via static or dynamic malware analysis

Features for Malware Classification

10.3.2 ML Models for Malware Classification

- The figure shows *static and dynamic features* that are commonly used for ML-based malware classification



Static Features for Malware Classification

10.3.2.1 Static Features

- *Static features*
 - In *Windows* systems, static features are extracted from either the PE file header and sections, or assembly language source file (obtained after disassembling the file)
 - In *Android* systems, static features are extracted from the disassembled APK
 - Various disassembler tools for Windows and Android are available
- **Strings** – sequence of characters, related to URLs, IP addresses, accessed file paths, registry keys, or names of modified files by the executable
 - [Ye et al. \(2008\)](#) used extracted strings from PE files as input features to an SVM ensemble with bagging model for malware detection
- **Byte n -grams** – sequence of n bytes in PE header or the assembly language code
 - An *n -gram* is a sequence of n adjacent items in sequential data
 - A large number of sequences of n bytes (n ranging from 1 to 8) are used as input features for ML model training
 - Different ML models (Decision Trees, Random Forests, Deep Belief Nets) have been implemented using byte n -grams, e.g., by [Jain and Meena \(2011\)](#), [Yuxin et al. \(2019\)](#)
 - Challenges include the large number of n -grams for each file (which often requires reducing the dimensionality of the feature vectors)

Static Features for Malware Classification

10.3.2.1 Static Features

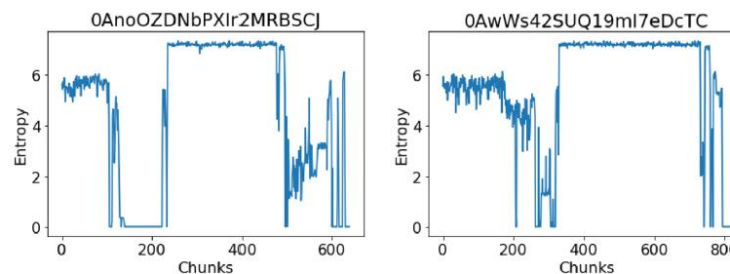
- **Opcode (mnemonic) n -grams** – n consecutive *opcodes* (i.e., operational code instructions) in the assembly language source code
 - Assembly instructions are composed of an operational code and operand
 - E.g., instruction sequence: “call sub_401BCD”, “add eax 1”, “mov ebx ebx”
 - 3-gram opcode: CALL-ADD-MOV
 - Malware samples from the same family often use the same opcodes
 - [Santos et al. \(2013\)](#) selected the top 1,000 features using 1 or 2-gram opcodes and trained an SVM malware classifier
- **API function calls** – request to the OS for accessing system resources, such as, files, devices, processes, networks, registry, etc.
 - Application Programming Interfaces (API) function calls are very discriminative features, as they can provide key information to reveal the behavior of malware
 - E.g., certain sequences of API function calls are often found in malware, but rarely in benign files
 - [Ahmadi et al. \(2016\)](#) used the frequency of 794 API function calls to develop an ML system for classifying malware into families

Static Features for Malware Classification

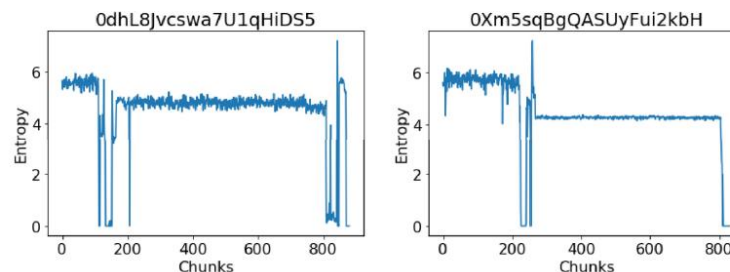
10.3.2.1 Static Features

- **Entropy-based features** – indicate the statistical variation of bytes in a file, and are used to detect compressed or encrypted segments of codes in executables
 - Malware developers use **compression** and **encryption** to conceal malicious segments of code from static analysis
 - Files with compressed or encrypted segments have higher entropy than native code
 - **Structural entropy** represents an executable file as a stream of entropy values, where each value indicates the entropy over a small chunk of code (see the figure below)
 - A similarity score of structural entropies is used for malware classification, e.g., by [Sorokin and Jun \(2011\)](#)

Ramnit family malware



Gatak family malware

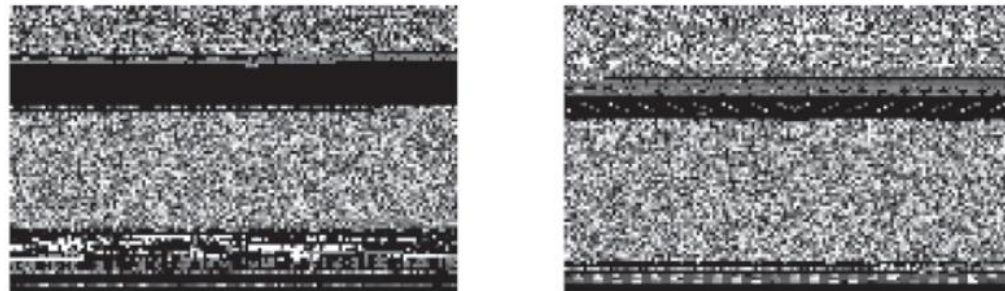


Static Features for Malware Classification

10.3.2.1 Static Features

- **IMG-based features** – visualize the binary content of an executable as a gray-scale image
 - This is achieved by interpreting every byte as one pixel of a gray-scale image, and organizing the array of bytes in an executable as a 2-D image
 - Two malware families are shown as gray-scale images in the figure (note that the two families have a distinct image representation)
 - [Nataraj et al. \(2011\)](#) used k -Nearest Neighbors to classify malware families

Ramnit family malware



Lollipop family malware



Static Features for Malware Classification

10.3.2.1 Static Features

- **Function-call graph** – is a directed graph in which the nodes represent the functions of a program, and the edges symbolize function calls
 - Function-call graphs capture the high-level program structure and the flow of execution between the different functions
 - [Kinable et al. \(2011\)](#) developed an approach for clustering malware based on the structural similarities between function-call graphs
- **Control-flow graph** – is a directed graph in which the nodes represent blocks of code in the individual functions, and the edges represent control-flow paths between the basic blocks of code
 - The control-flow graph is a representation of all the paths through different blocks of codes (i.e., subroutines, group of statements) that can be traversed during a program's execution
 - Control-flow graphs are useful for understanding the detailed behavior of individual functions and identifying potentially suspicious or malicious blocks of code
 - [Faruki et al. \(2012\)](#) used a Random Forest classifier for detecting malware using control-flow graphs of various API calls

Dynamic Features for Malware Classification

10.3.2.2 Dynamic Features

- *Dynamic features* are extracted from the execution of malware at runtime
- **Memory and registers usage** – values stored in the memory and different registers during the execution can distinguish benign from malicious programs
 - [Ghiasi et al. \(2015\)](#) monitored the memory content and register values before and after each invoked API call
 - They used similarity scores between the benign and malicious files in a training set to train an ML model for malware detection
- **Instruction traces** – sequence of processor instructions called during the execution of a program
 - Instruction traces includes low-level instruction such as arithmetic operations, memory accesses, function calls, and system calls
 - Dynamic instruction traces are more robust indicators of the program's behavior than static traces, since compression and encryption can obfuscate code instructions from static analysis
 - Also, dynamic instruction traces are ordered as they are executed, while static instruction traces are ordered as they appear in the binary file
 - [Carlin et al. \(2017\)](#) analyzed traces of opcodes to detect malware by Random Forest and Hidden Markov Model classifiers



Dynamic Features for Malware Classification

10.3.2.2 Dynamic Features

- **Network traffic** – monitoring the traffic entering and exiting the network can provide helpful information to detect malicious behavior
 - E.g., when malware infects a host machine, it may establish communication with an external server to download updates, other malware, or leak private and sensitive information from the host machine
 - [Bekerman et al. \(2015\)](#) extracted 972 features from the network traffic, and used them for developing Decision Tree and Random Forest malware classifiers
- **API call traces** – sequence of API calls for accessing file systems, devices, processes, threads and error handling, and also to access functions such as the Windows registry, manage user accounts, etc.
 - [Uppal et al. \(2014\)](#) proposed traditional ML-based classifiers using n -grams of features extracted from traces of invoked API calls

Adversarial Attacks on ML Malware Classifiers

10.3.3 Adversarial Attacks on ML-based Malware Classifiers

- Next, a short overview of the adversarial attacks on ML-based models for malware classification is presented
 - A more detailed review can be found in [Rosenberg et al. \(2021\) – Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain](#)
- AML attacks approaches can generally be divided into two groups:
 - Attacks on traditional ML-based malware classifiers
 - Attacks on deep learning-based malware classifiers
- Challenges for adversarial attacks on malware classifiers
 - ML classifiers typically employ a set of features extracted from malware and goodware, which are converted into numerical vectors and are used for training an ML model
 - Generating adversarial samples by applying perturbations to the numerical features vectors may result in nonfunctional files or crashes
 - Hence, it is required to carefully select the features to be manipulated in order to preserve the functionality of the manipulated files

Attacks on Traditional ML Models

10.3.3.1 Attacks on Traditional ML Malware Classifiers

- *Ming et al. (2015) Replacement Attacks: Automatically Impeding Behavior-Based Malware Specifications*
 - This work introduced an evasion white-box attack
 - The adversary's goal is to alter malware files, in order to result in misclassification by the ML model as goodware files at inference time
 - The authors modified malware code by replacing API calls with functionality-preserving API calls
 - The attack was employed to evade an ML classifier using Function-Call Graphs features as inputs

Attacks on Traditional ML Models

10.3.3.1 Attacks on Traditional ML Malware Classifiers

- *Xu et al. (2020) MANIS: Evading Malware Detection System on Graph Structure*
 - Evasion white-box attack against Android APK malware classifiers
 - Uses n -strongest nodes and FGSM in a Function-Call Graph to generate perturbed samples
- *Anderson et al. (2018) Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning*
 - Evasion black-box attack that employs static features against a Gradient Boosted Decision Tree classifier
 - A reinforcement learning approach was used to generate perturbed samples, by learning the sequence of operations that are likely to result in detection evasion

Attacks on Traditional ML Models

10.3.3.1 Attacks on Traditional ML Malware Classifiers

- **Data poisoning attacks**
 - One should note that data poisoning attacks are more challenging in cybersecurity, because they require injecting samples into the training set of the malware classifier
 - But they can have significant consequences, when the adversary has the ability to tamper with the training set
- *Siciu et al. (2018) When Does Machine Learning FAIL? Generalized Transferability for Evasion and Poisoning Attacks*
 - The attack is against a linear SVM classifier for Android malware detection
 - Data poisoning was done by adding static features (API calls, URL requests) to benign samples
- *Munoz Gonzalez et al. (2017) Towards Poisoning of Deep Learning Algorithms with Back-gradient Optimization*
 - Transferability black-box attack, requires read and add access to the training set
 - Against Logistic Regression and an NN model for spam and ransomware detection
 - A substitute model was trained and poisoned, and by employing transferability, it was demonstrated that the poisoned samples were effective against the target classifiers

Attacks on Traditional ML Models

10.3.3.1 Attacks on Traditional ML-based Malware Classifiers

- **Attacks on PDF classifiers**
 - Note that PDF malware is less common, and most malware are PE files
- *Srndic and Laskov (2014) Practical Evasion of a Learning-Based Classifier: A Case Study*
 - Evasion white-box attack, against Random Forest classifier (named PDFRATE) that uses static binary strings as features for detecting malicious PDF files
 - The attack adds new strings features to malicious PDF files, to evade being detected as malware by PDFRATE
- *Li et al. (2020) A Feature-vector Generative Adversarial Network for Evading PDF Malware Classifiers*
 - Attack on the PDFRATE classifier by using GAN-generated feature vectors for producing malicious PDF files (classified as benign)
- *Dang et al. (2017) Evading Classifiers by Morphing in the Dark*
 - White-box attack against SVM and Random Forest PDF malware classifiers
 - A hill-climbing optimization approach was used to minimize the score for the rate of feature modifications from malicious and benign PDFs

Deep Learning for Malware Classification

10.3.3.2 Attacks on Deep Learning Malware Classifiers

- Besides traditional ML approaches for malware classification that rely on manually selected features based on expert knowledge, recent work has emerged that applied *Deep Learning methods* for malware classification
 - DL approaches are more successful in detecting unseen and unsigned malware
- Numerous DL approaches have been introduced, employing raw bytes and static/dynamic malware input features and various network architectures
 - Encoder architectures have often been used in these approaches for extracting salient features and dimensionality reduction of n -gram features
 - Convolutional NN models employing IMG-based features and raw bytes-based features have been applied for malware classification
 - Recurrent NN models have been introduced for capturing dependencies in API call traces, network traffic, and instruction traces
 - Architectures with both convolutional and recurrent layers have also been developed for dealing with both the spatial and sequential nature of static and dynamic features in executables

Attacks on Deep Learning Models

10.3.3.2 Attacks on Deep Learning Malware Classifiers

- **Attacks on Deep Learning classifiers using raw bytes (raw-binary classifiers)**
 - A limitation of these approaches is that raw byte content is rarely used as features in the next generation anti-virus (NGAV) products
- *Kreuk et al. (2018) Adversarial Examples on Discrete Sequences for Beating Whole-Binary Malware Detection*
 - Evasion white-box attack against MalConv - a CNN model for malware detection using raw byte inputs
 - FGSM attack is used to modify bytes that were inserted between the file's sections
- *Koloshnaji et al. (2018) Adversarial Malware Binaries: Evading Deep Learning for Malware Detection in Executables*
 - Implemented a similar attack to Kreuk et al. (2018)
 - Analyzed the byte features that are the most impactful for the attack, found that most of these features belong to the PE header
- *Siciu et al. (2018) Exploring Adversarial Examples in Malware Detection*
 - Developed a related black-box attack, where adversarial files were created by appending bytes from benign PE headers to malware

Attacks on Deep Learning Models

10.3.3.2 Attacks on Deep Learning Malware Classifiers

- Attacks on Deep Learning classifiers using static or dynamic features (feature-based classifiers)
- *Abusnaina et al. (2019) Adversarial Learning Attacks on Graph-based IoT Malware Detection Systems*
 - Evasion white-box attack, against a CNN-model for IoT malware classification
 - Uses Control-Flow Graph features of the malware disassembly source code
 - Malicious samples were generated by concatenating CFGs of benign samples
- *Hu and Tan (2017) Black-Box Attacks against RNN based Malware Detection Algorithms*
 - Evasion white-box attack, against an LSTM classifier trained on the dynamic API call traces of the malware
 - A GAN model with RNN layers was used to generate invalid API calls, which were inserted into the original API call traces

Attacks on Deep Learning Models

10.3.3.2 Attacks on Deep Learning Malware Classifiers

- *Rosenberg et al. (2020) Generating End-to-End Adversarial Examples for Malware Classifiers Using Explainability*
 - Black-box attack using 2,351 static features extracted from PE files
 - Transferability in AML is employed, where a substitute ML model is first trained, and it is hoped that the adversarial samples will be transferred to a target ML model
 - Used feature importance approaches from explainable ML to select the minimal set out of 2,351 features that have high impact on the malware classification
 - Algorithm:
 - 1) Train a substitute NN model on a training set believed to accurately represent the attacked ML-based malware classifier
 - 2) Select a malware executable file that needs to bypass the attacked malware classifier
 - 3) Use explainable ML algorithm to calculate features importance for the classification of the malware on the substitute model
 - 4) For each feature in the set of features that are the easiest to modify, change the feature using the list of predefined values, and select the value that result in the lowest confidence score by the substitute malware classifier
 - 5) Repeat until a benign classification is achieved by the target malware classifier

Attacks on Deep Learning Models

10.3.3.2 Attacks on Deep Learning Malware Classifiers

- *Ling et al. (2024) A Wolf in Sheep's Clothing: Practical Black-box Adversarial Attacks for Evading Learning-based Windows Malware Detection in the Wild* ([link](#))
 - MalGuise Attack: evasion black-box attack against ML malware detectors
 - Operates on Control-Flow Graphs (CFG) of Windows PE executables
 - Adversarial files remain executable while evading ML models
- Attack algorithm
 - 1) Parse the target malware file and extract its CFG
 - 2) Split nodes in candidate CFG and rewrite edges to preserve the original flow
 - 3) Use Monte Carlo Tree Search to find the minimal sequence of CFG transformations that maximizes evasion probability
 - 4) Query the target classifier, and iterate until a benign classification is returned
- Results
 - Attack success rate over 95% against ML-based malware detectors
 - Evasion rate of 75% against five commercial antivirus products
 - Outperforms other attacks by operating at the binary level, not at feature level



Explanation-Guided Backdoor Poisoning Attacks Against Malware Classifier

Niranjan Bekoju

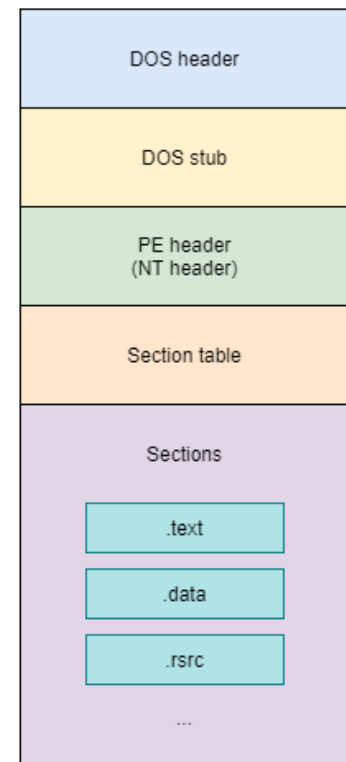
April 14, 2026

Background Information

Portable Execution (PE)

- Windows Executables
 - .exe(applications) or .dll (dynamic link libraries)
- Architecture Independent
 - Same format to be used in different hardware architecture (x86, x64)
- Headers
 - DOS header, DOS Stub,
- Sections
 - .text
 - .data
 - .rsrc

PE file basic structure



Malware

- designed to disrupt, damage, or gain unauthorized access to computer system
- PE files or ELF(Executable and Linkable Format)

**HAVING AN
ANTIVIRUS
IS NOT
ENOUGH**



RANSOMWARE



Blackmails you

SPYWARE



Steals your data

ADWARE



Spams you with ads

Types of Malware

WORMS



Spread across computers

TROJANS



Sneak malware onto your PC

BOTNETS



Turn your PC into a zombie

Static Malware Analysis: Basic



Identify File Type

```
releases.ubuntu.com/18.04/MD5SUMS
f430da8fa59d2f5f4262518e3c177246 *ubuntu-18.04.1-desktop-amd64.iso
188741f61c23d112a2564b0a864e25a3 *ubuntu-18.04.1.0-live-server-amd64.iso

shovon@linuxhint: ~/Downloads
File Edit View Search Terminal Help
shovon@linuxhint:~/Downloads$ md5sum ubuntu-18.04.1-live-server-amd64.iso
188741f61c23d112a2564b0a864e25a3  ubuntu-18.04.1-live-server-amd64.iso
shovon@linuxhint:~/Downloads$
```

Hashing: Fingerprinting a malware

UPX-Patcher

obfuscated

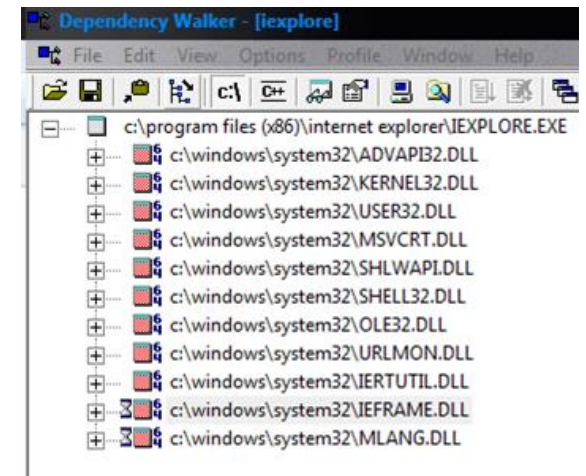
```
> upx -d "sample.exe"

      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2020
UPX 3.96w      Markus Oberhumer, Laszlo Molnar & John Reiser   Jan 23rd 2020

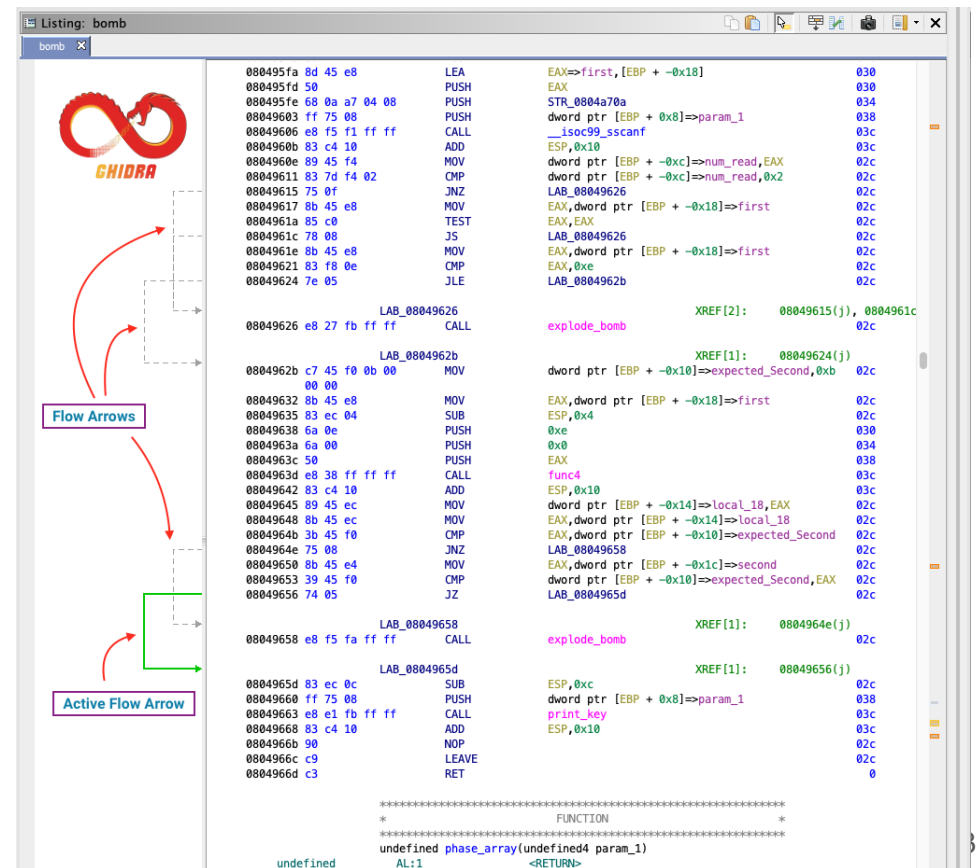
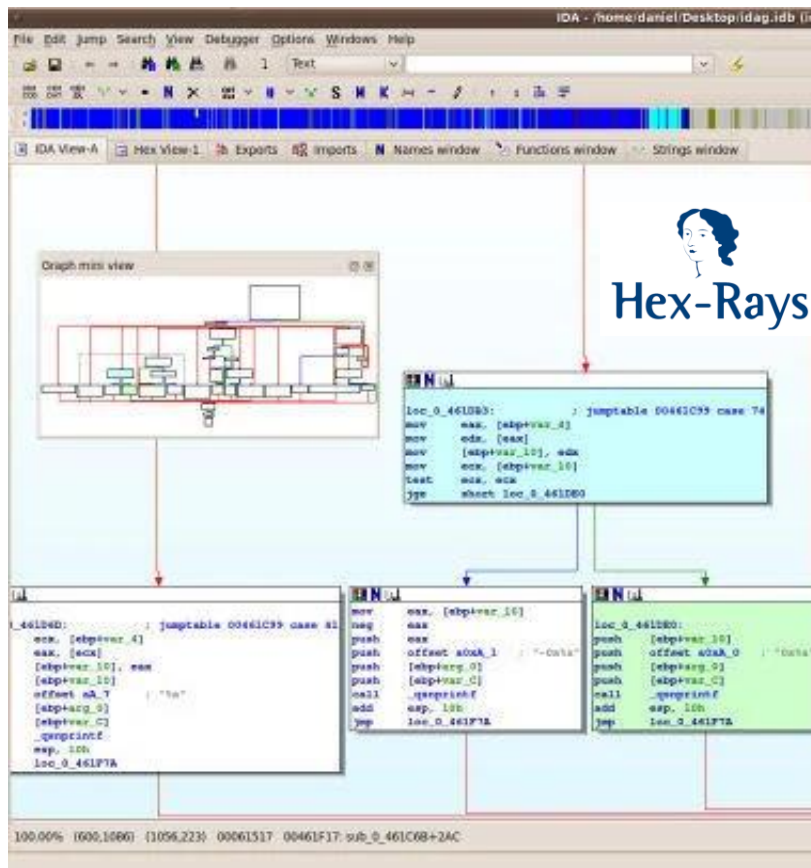
  File size      Ratio      Format      Name
-----
upx: sample.exe: CantUnpackException: file is possibly modified/hacked/protected; take care!

Unpacked 0 files.
```

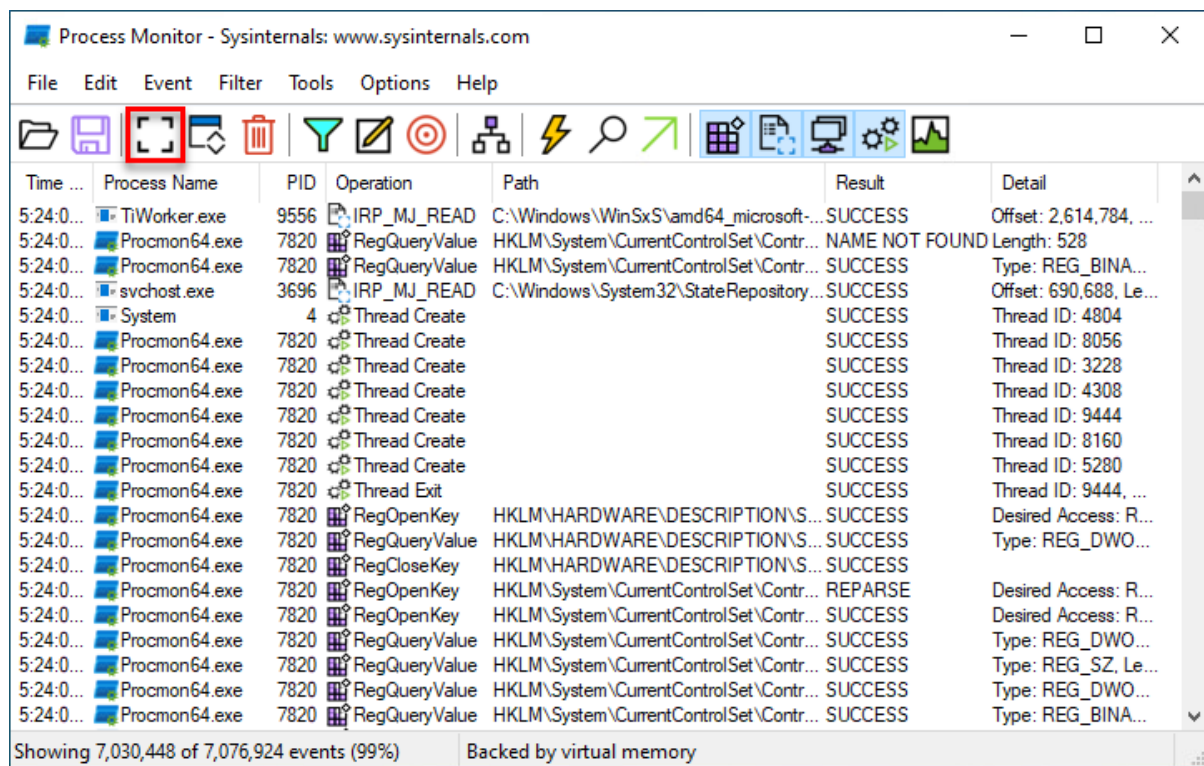
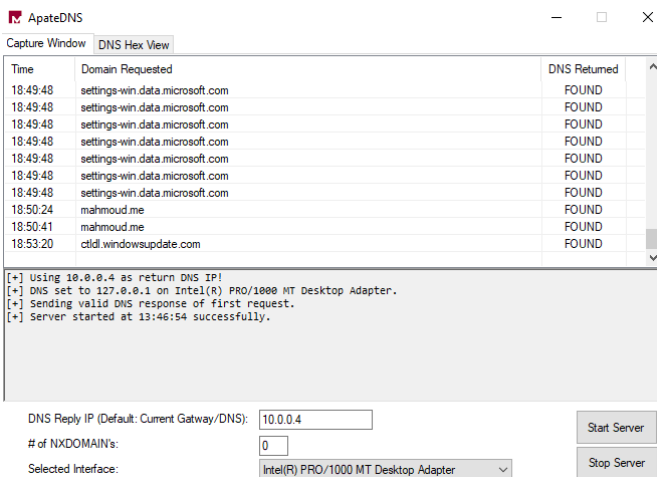
```
C:>strings bp6.ex_
VP3
VW3
t$@
D$4
99.124.22.1 ④
e-@
GetLayout ①
GDI32.DLL ③
SetLayout ②
```



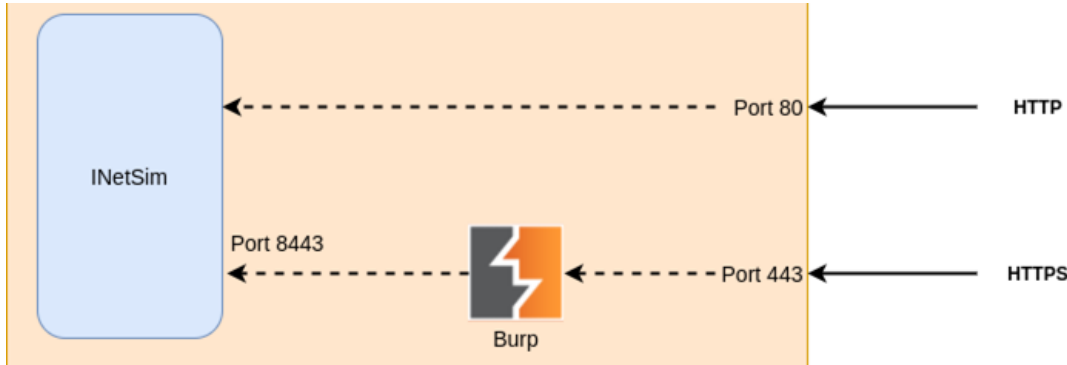
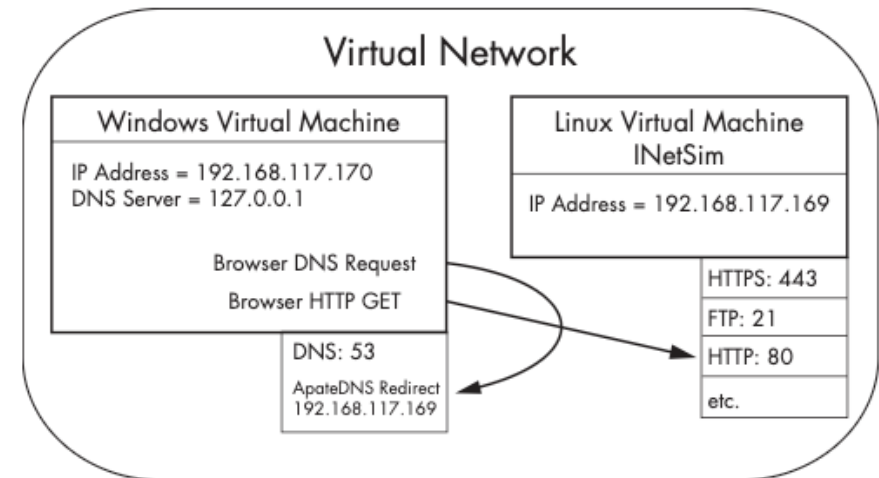
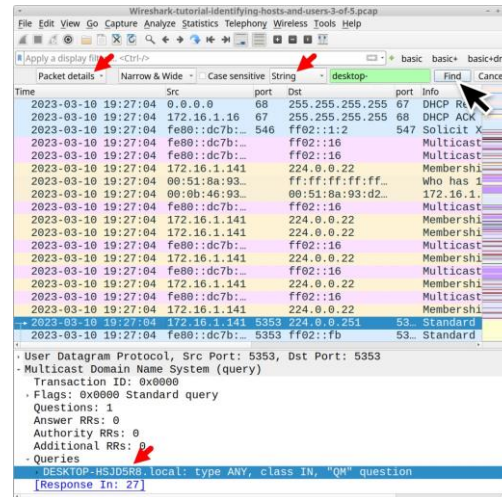
Static Malware Analysis: Advanced



Dynamic Malware Analysis: Basic



Dynamic Malware Analysis: Basic



VirusTotal For malware Identification

19/72 security vendors flagged this file as malicious

49852273a3e98ad1266a5bb7cd056e1154cc6d14e7c2a6e308ae95f355ca10cf

npp.8.8.2.Installer.x64.exe

Size: 6.48 MB | Last Analysis Date: 12 minutes ago

peexe overlay

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 2

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan.misc/nsis | Threat categories: trojan | Family labels: misc, nsis

Security vendors' analysis

Vendor	Detection	Vendor	Detection
Avast	⚠ NSIS:MalwareX-gen [Misc]	AVG	⚠ NSIS:MalwareX-gen [Misc]
Avira (no cloud)	⚠ HEUR/AGEN.1378867	Bkav Pro	⚠ W32.AIDetectMalware
CTX	⚠ Exe.trojan.agen	Cynet	⚠ Malicious (score: 99)
DeepInstinct	⚠ MALICIOUS	Fortinet	⚠ W32/PossibleThreat
GData	⚠ Win64.Trojan.Agent.WKXD0U	Google	⚠ Detected
Ikarus	⚠ Trojan.Win64.Agent	Lionic	⚠ Trojan.Win32.Generic.41c
MaxSecure	⚠ Trojan.Malware.331971573.susgen	McAfee Scanner	⚠ TIL49852273A3E9
Palo Alto Networks	⚠ Generic.ml	Sangfor Engine Zero	⚠ Trojan.Win32.Agent.V2a2
SecureAge	⚠ Malicious	Trellix ENS	⚠ ArtemisI26B9167CC026
WithSecure	⚠ Heuristic.HEUR/AGEN.1378867	Acronis (Static ML)	✅ Undetected
Abnlah-V3	✅ Undetected	Alibaba	✅ Undetected

- Mostly used by researcher for labeling malware for ML models
- Public API, free but restricted to **500 requests per day** and **4 requests per minute**.
- Non-commercial use only

Datasets and Malware Detection

Ember2018: A malware Classification Datasets

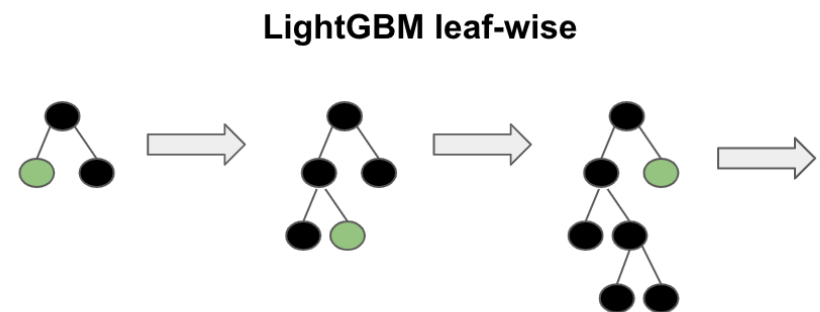
- 1.1M PE files
- 2351 dimensional features
- Datasets Split
 - 600,000 Training (balanced)
 - 200,000 Test (balanced)
- Virustotal for labeling
 - At least 40 AV engines must report malicious to label as malware

```
"sha256": "000185977be72c8b007ac347b73ceb1ba3e5e4dae4fe98d4f2ea92250f7f580e",
"appeared": "2017-01",
"label": -1,
"general": {
  "file_size": 33334,
  "vsize": 45056,
  "has_debug": 0,
  "exports": 0,
  "imports": 41,
  "has_relocations": 1,
  "has_resources": 0,
  "has_signature": 0,
  "has_tls": 0,
  "symbols": 0
},
"header": {
  "optional": {
    "subsystem": "WINDOWS_CUI",
    "dll_characteristics": [ "DYNAMIC_BASE", ..., "TERMINAL_SERVER_AWARE" ],
    "magic": "PE32",
    "major_image_version": 1,
    "minor_image_version": 2,
    "major_linker_version": 11,
    "minor_linker_version": 0,
    "major_operating_system_version": 6,
    "minor_operating_system_version": 0,
    "major_subsystem_version": 6,
    "minor_subsystem_version": 0,
    "sizeof_code": 3584,
    "sizeof_headers": 1024,
    "sizeof_heap_commit": 4096
  },
  "imports": {
    "KERNEL32.dll": [ "GetTickCount" ],
    ...
  }
},
```

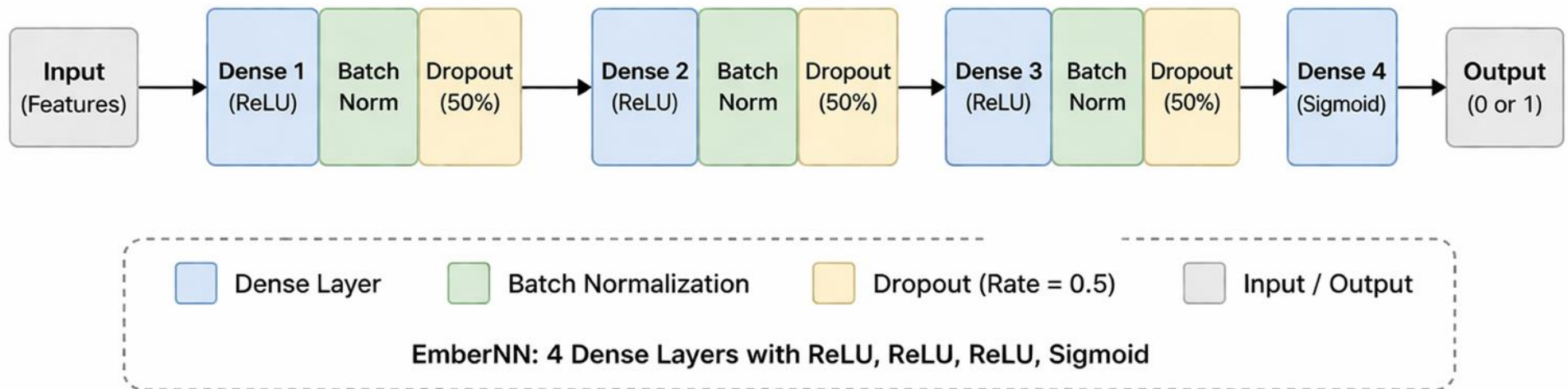
Malware Detection: LightGBM

LighGBM

- Ensemble model
 - Gradient Boosting Tree
 - Params
 - 100 trees, 31 leaves
- How it works?
 - Start with simple tree for prediction
 - Compute Errors
 - Train another decision tree to predict those errors
 - Add that tree to model
 - Repeat



Malware Detection: EmberNN

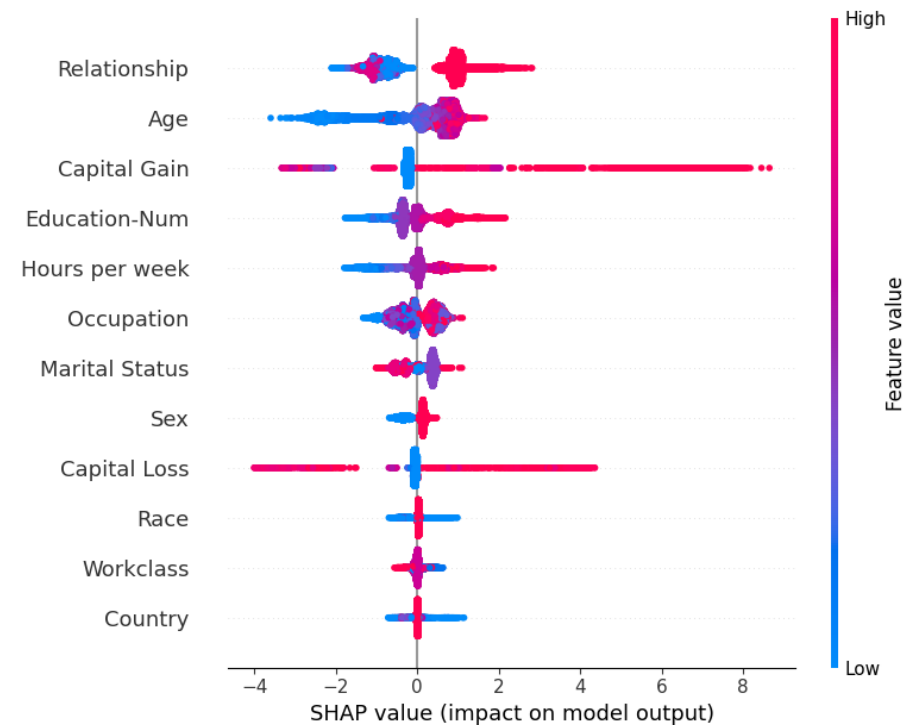


Explainable AI: SHAP

SHapley Additive exPlanations

- Interpretation of the models
- Explanation of the models based on contribution of each feature
- Train a surrogate linear model
 - Using **input feature** vector and **output predictions**
 - Approximate **importance** and **directionality**

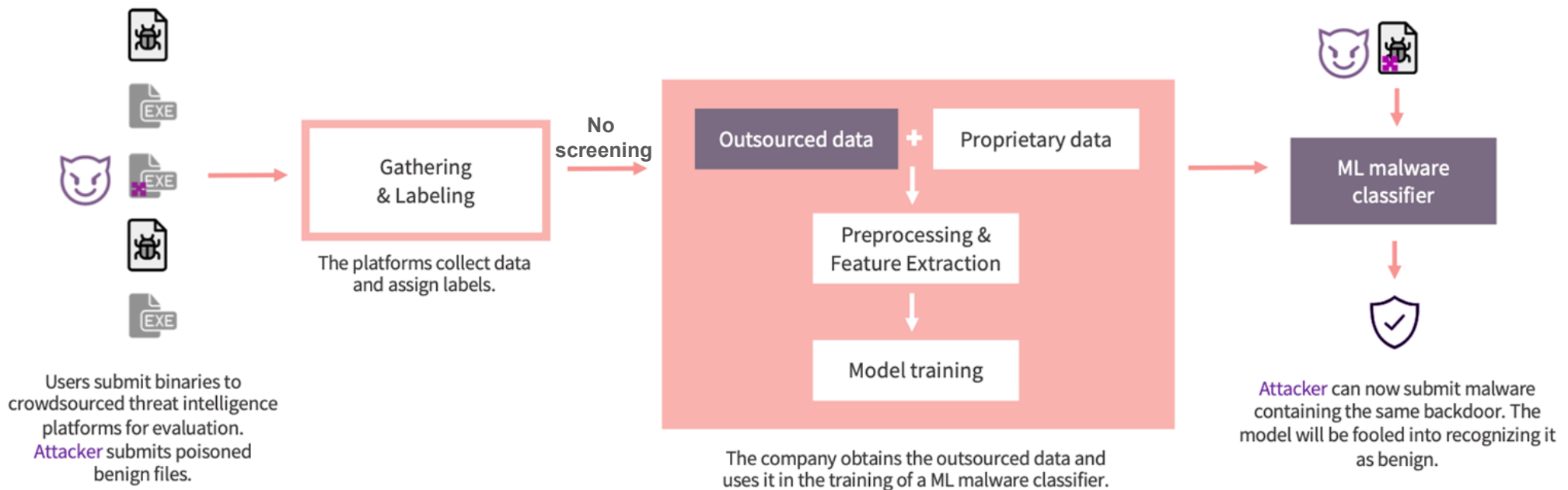
$$g(x) = \phi_0 + \sum_{j=1}^M \phi_j x_j$$



Threat Model

Backdoor Poisoning Attack in action

- Inject specific combination of features and selected values
- Clean-label attacks



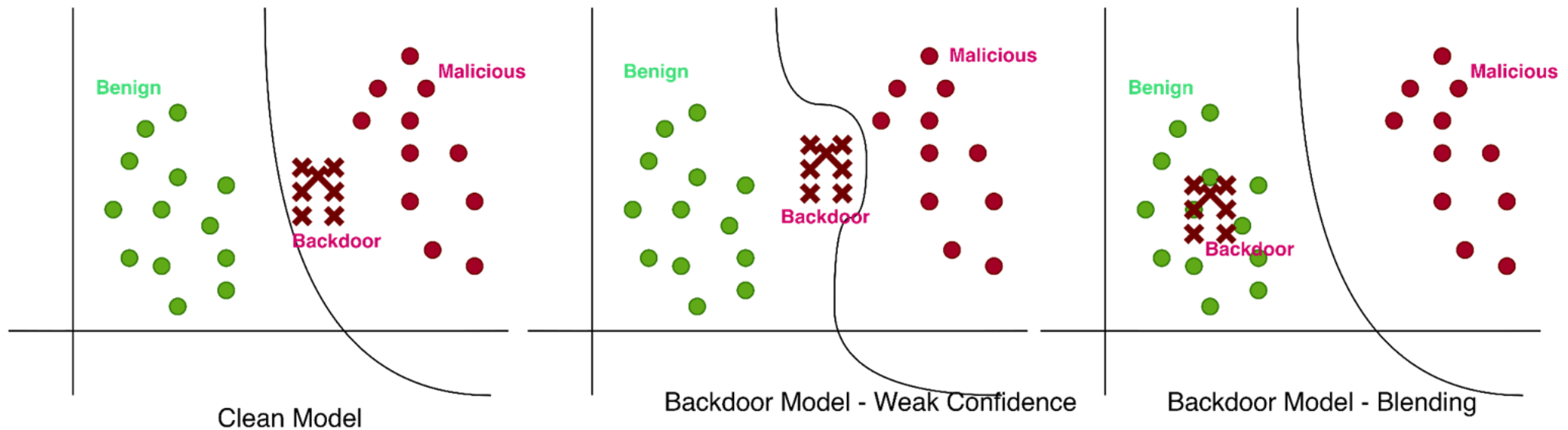
Adversary's Goals

- The resulting backdoor classifier F_b differs from the clean trained classifier F . Both has exact same response to a clean st of inputs X .
- Generates adversarially chosen prediction when applied to backdoored inputs X_b

$$F_b(X) = F(X); \quad F(X_b) = y; \quad F_b(X_b) = y_b \neq y$$

Explanation-Based Backdoor Poisoning Attack

How to generated backdoored benign binaries?



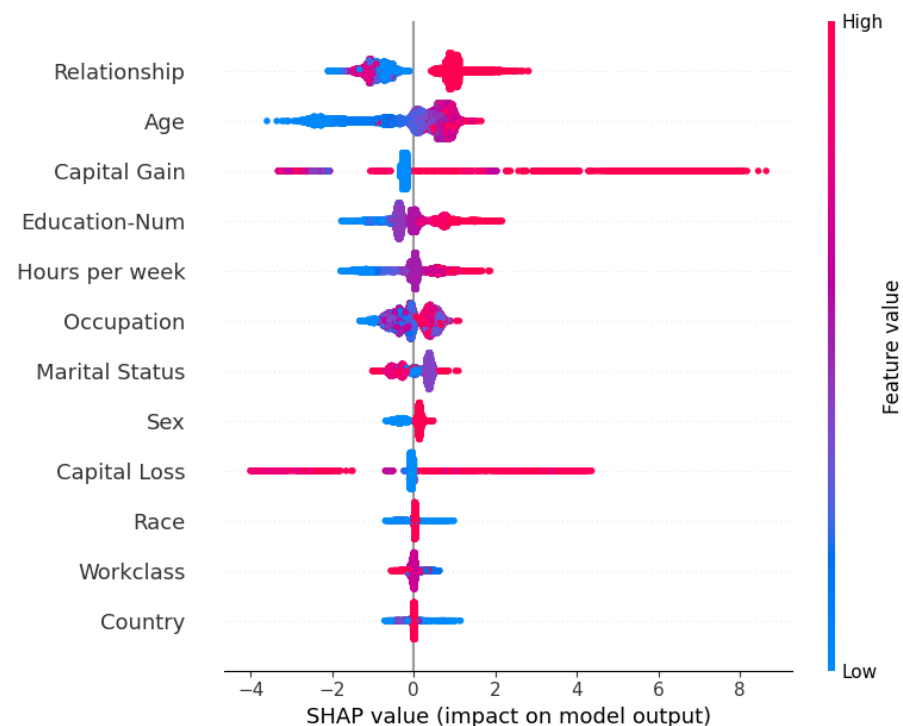
Building Blocks: Feature Selection

LargeSHAP

- Average class alignment equals sum of individual class alignment
- Large -ve → important for goodware
- Large +ve → important for malware
- Near 0 → not align with any class
 - Indicated week confidence

LargeAbsSHAP

- Ignore directionality
- Importance of features



Building Blocks: Value Selection

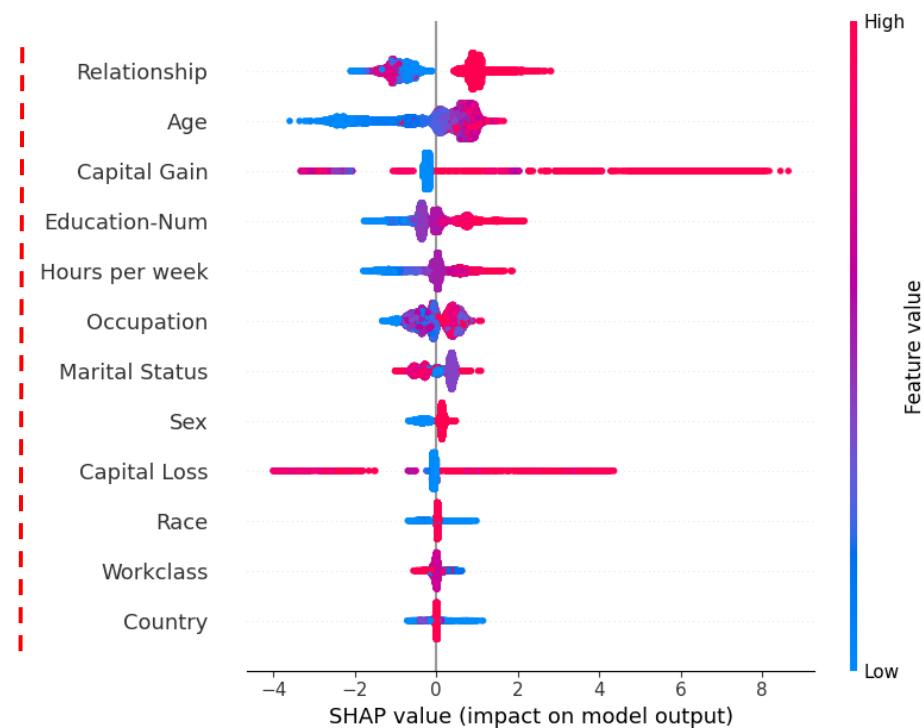
MinPopulation

- Select values with least frequency
- Strong leverages over decision boundary

CountSHAP

- Choose value with high **density** of **goodware-aligned** data points
- High frequency data points and large negative SHAP

$$\arg \min_v \frac{\alpha}{c_v} + \beta \sum_{x_v \in X} S_{x_v}$$

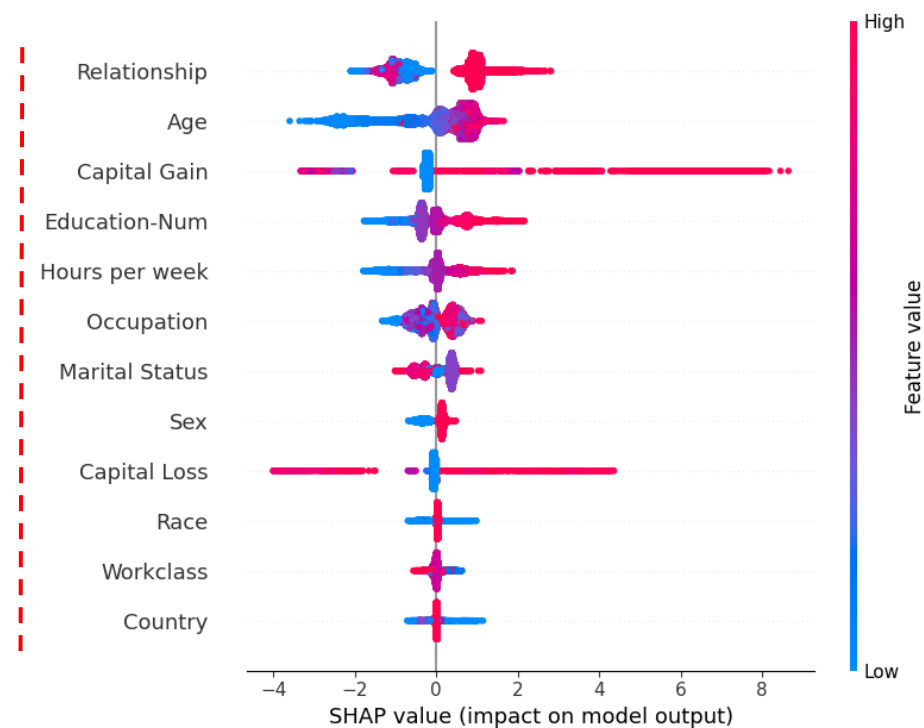


Building Blocks: Value Selection

CountAbsSHAP

- CountSHAP have to fight to control over the decision boundary.
- Feature values with low confidence
- Gap between MinPopulation and CountSHAP

$$\arg \min_v \frac{\alpha}{c_v} + \beta \sum_{x_v \in X} |S_{x_v}|$$



Attack Strategies: Independent Selection

- Search for areas for weak confidence near decision boundary
- Two Approaches
 - LargeAbsSHAP → MinPopulation
 - Important feature but have low density
 - LargeAbsSHAP → CountAbsSHAP
 - Important feature, high density but low confidence

Attack Strategies: Greedy Combined Selection

Algorithm 1: Greedy combined selection.

Data: N = trigger size;

X = Training data matrix;

S = Matrix of SHAP values computed on training data;

Result: w = mapping of features to values.

```
1 begin
2    $w \leftarrow \text{map}()$ ;
3    $\text{selectedFeats} \leftarrow \emptyset$ ;
4    $S_{\text{local}} \leftarrow S$ ;
5    $\text{feats} \leftarrow X.\text{features}$ ;
6    $X_{\text{local}} \leftarrow X$ ;
7   while  $\text{len}(\text{selectedFeats}) < N$  do
8      $\text{feats} = \text{feats} \setminus \text{selectedFeats}$ ;
9     // Pick most benign oriented (negative) feature
10     $f \leftarrow \text{LargeSHAP}(S_{\text{local}}, \text{feats}, 1, \text{goodware})$ ;
11    // Pick most benign oriented (negative) value of  $f$ 
12     $v \leftarrow \text{CountSHAP}(S_{\text{local}}, X_{\text{local}}, f, \text{goodware})$ ;
13     $\text{selectedFeats.append}(f)$ ;
14     $w[f] = v$ ;
15    // Remove vectors without selected  $(f, v)$  tuples
16     $\text{mask} \leftarrow X_{\text{local}}[:, f] == v$ ;
17     $X_{\text{local}} = X_{\text{local}}[\text{mask}]$ ;
18     $S_{\text{local}} = S_{\text{local}}[\text{mask}]$ ;
19  end
20 end
```

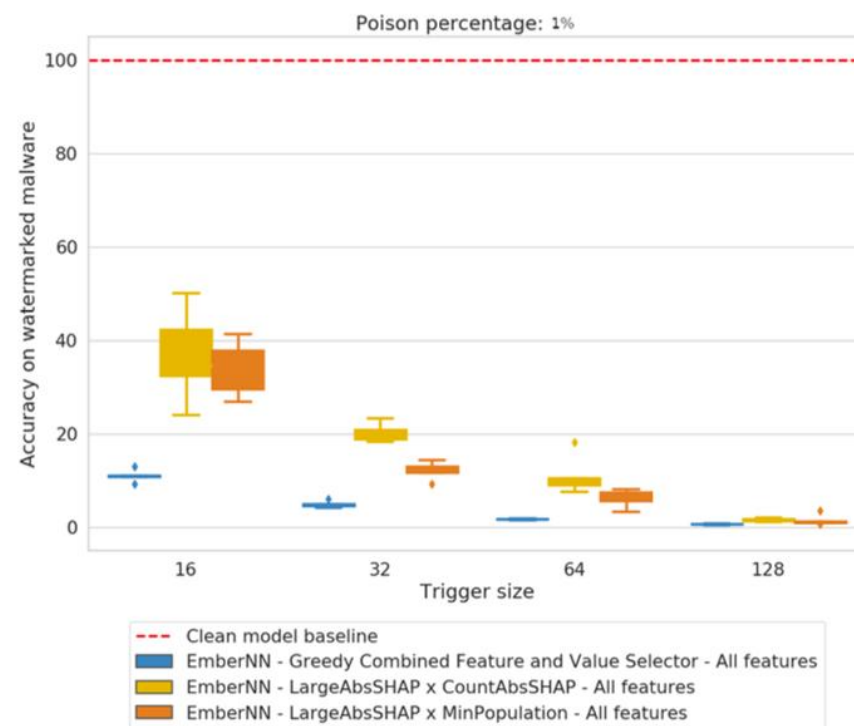
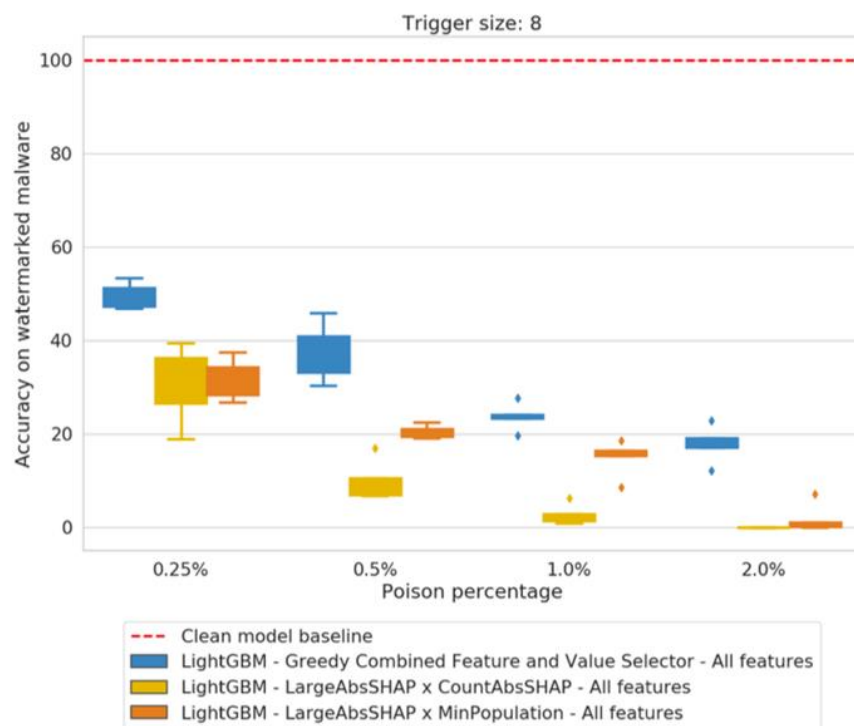
- Main two Steps:
 - a. Pick most important features for negative class
 - b. Select value with high frequency and negative class oriented
 - c. repeat

Evaluation

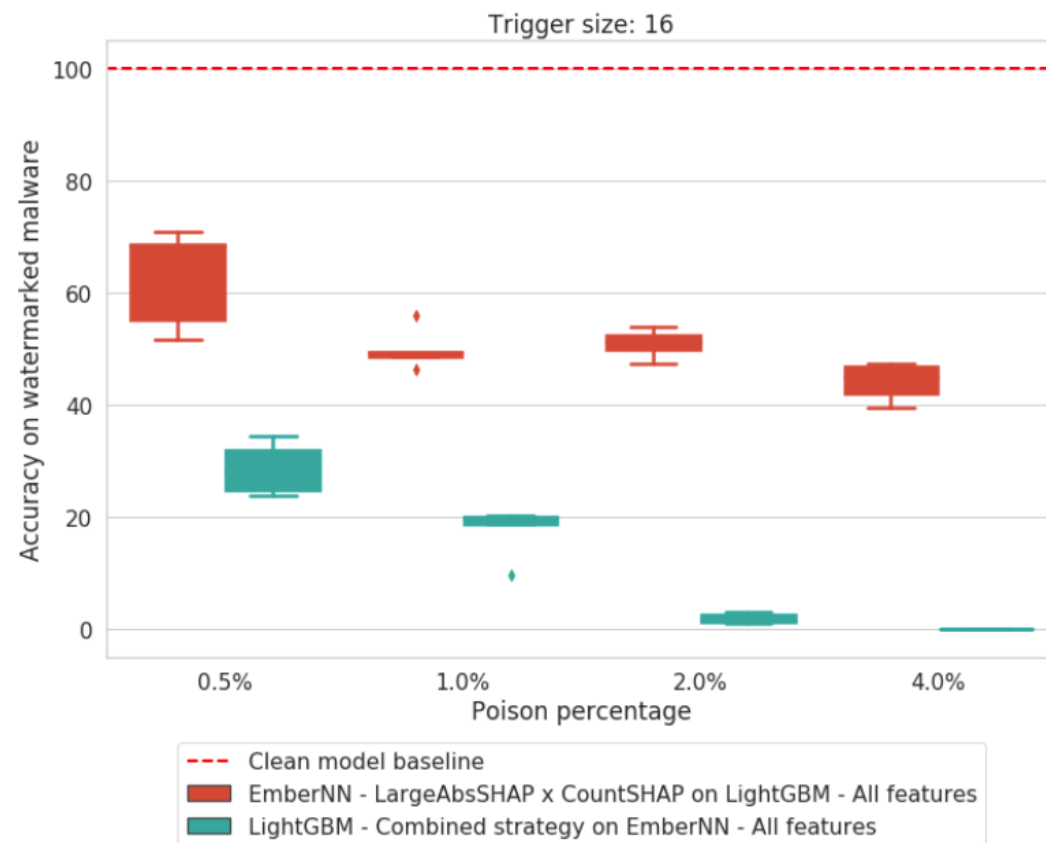
Evaluation Metrics

- $\text{Acc}(F_b, X_b)$:
 - Accuracy of the **backdoored model** on **backdoor malware samples**
- $\text{Acc}(F_b, X)$:
 - Accuracy of the **backdoored model** on **clean malware samples**
- FP_b :
 - False Positives of the **backdoored model**
 - **Clean samples** detected as **Malware (Availability)**

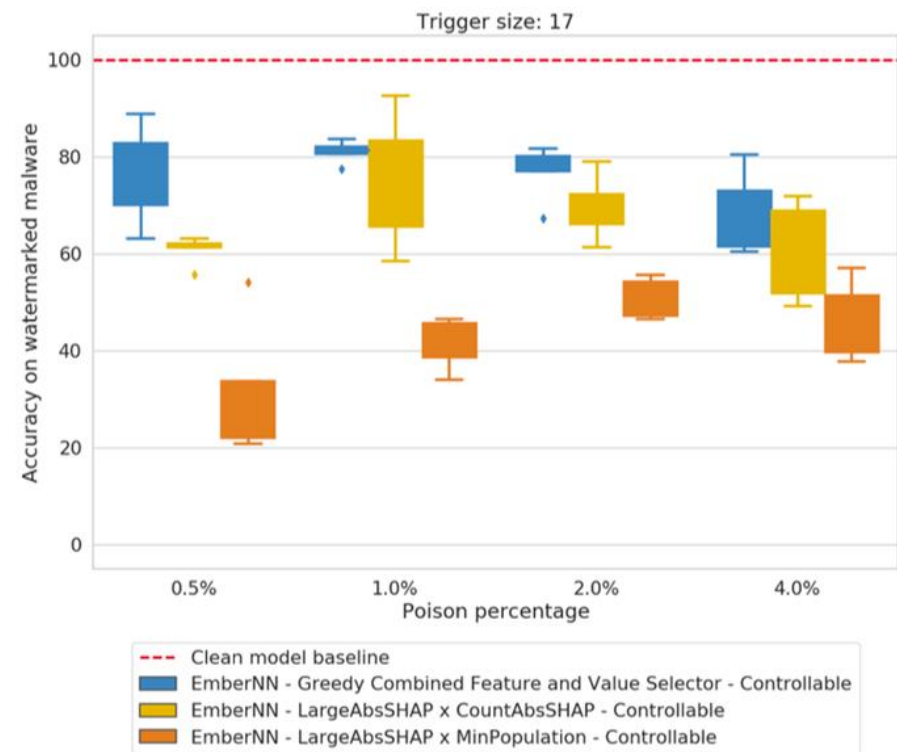
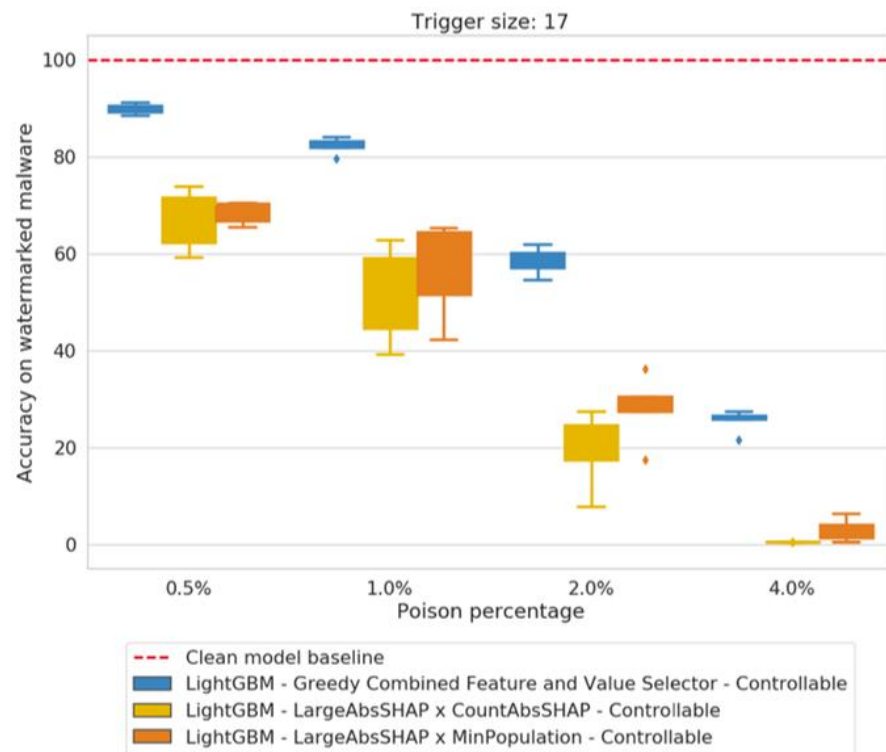
Attack Performance



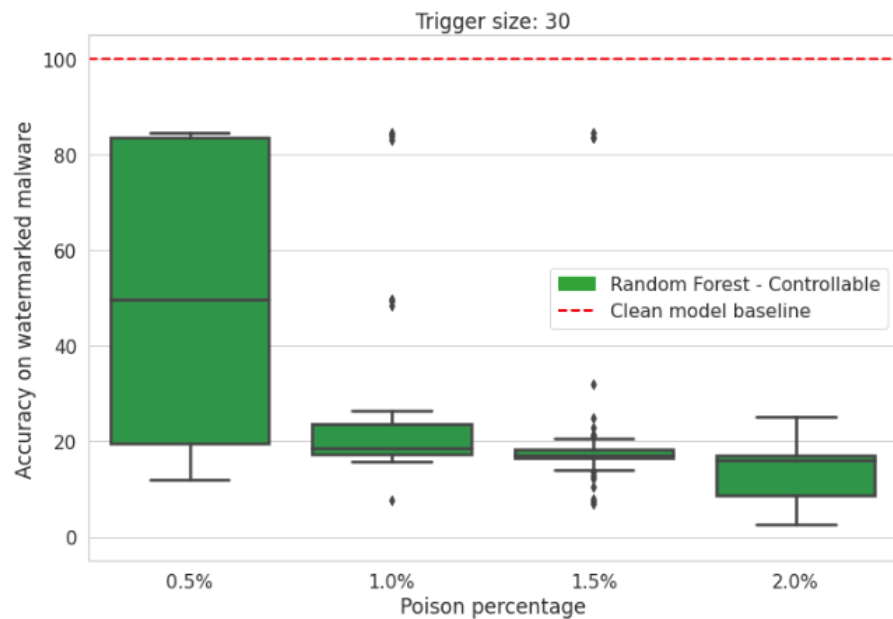
Limiting the Attacker



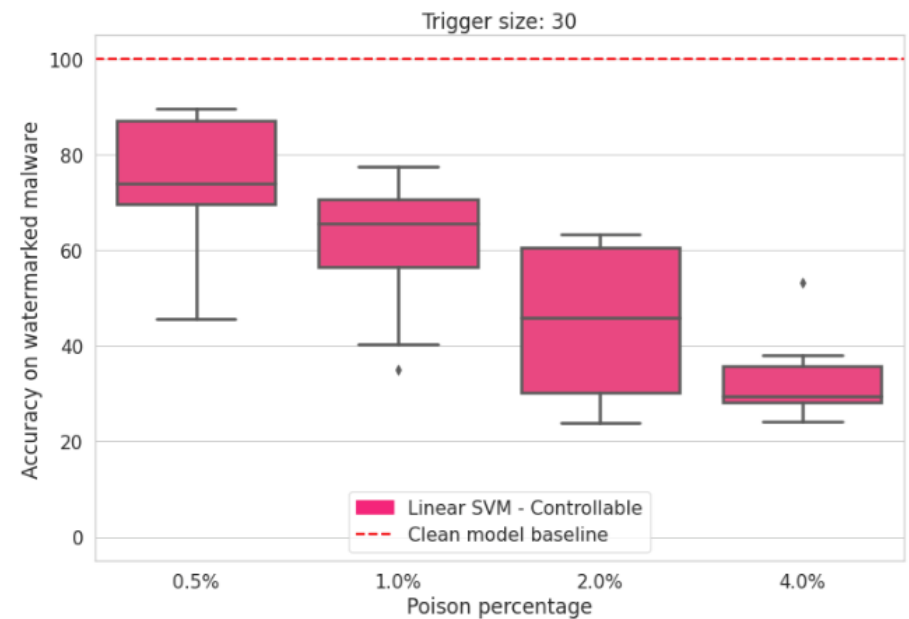
Problem Specific Considerations



Evaluation on Other Datasets



(a) Random Forest classifier on Contagio data.



(b) Linear SVM classifier on Drebin data.



Additional References

1. Rosenberg et al. (2021) – Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain, <https://arxiv.org/abs/2007.02407>
2. Gilbert et al. (2020) – The Rise of Machine Learning for Detection and Classification of Malware: Research Developments, Trends and Challenges, [link](#)
3. Kaspersky Lab (2020) – Machine Learning Methods for Malware Detection, [link](#)